

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Roll No. A24126552264

Student Name K.Hansikha

1. TITLE

Develop Dynamic Webpage Using DOM & JS

2. AIM

To develop a dynamic webpage using JavaScript and the Document Object Model (DOM) to add and delete tasks.

3. OBJECTIVES

1. To understand the Document Object Model.
2. To select and modify HTML elements using JavaScript.
3. To handle user events such as button clicks.
4. To create new DOM elements dynamically.
5. To remove elements from the webpage using JavaScript.

4. THEORY

The DOM represents an HTML document as a tree of objects. JavaScript can access these objects, change their content and attributes, create new nodes and remove existing nodes. Event listeners connect user actions such as clicks to these DOM operations.

5. ALGORITHM/PROCEDURE

1. Create a webpage containing an input field, button and task list.
2. Select the input and list elements using DOM methods.
3. Attach a click event listener to the Add Task button.
4. Read the task text from the input field.
5. Create a new list item and delete button dynamically.
6. Append the new task to the task list.
7. Attach a delete event to remove the selected task.
8. Open the page and verify dynamic interaction.

6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Task Manager</title>
<style>
body{font-family:Arial;background:#eef2f7;padding:35px}
.app{max-width:650px;margin:auto;background:white;padding:30px;border-radius:14px}
input{padding:11px;width:70%}
button{padding:10px 14px;margin-left:6px}
li{margin:15px 0;font-size:18px}
.delete{margin-left:12px}
</style>
</head>
<body>
<div class="app">
<h1>Dynamic Task Manager</h1>
<input id="taskInput" placeholder="Enter a task">
<button id="addBtn">Add Task</button>
<ul id="taskList">
<li>Complete DOM experiment <button class="delete">Delete</button></li>
<li>Practice JavaScript events <button class="delete">Delete</button></li>
</ul>
</div>
<script>
const input = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn");
const list = document.getElementById("taskList");
function addTask(){
  const text = input.value.trim();
  if(text === "") return;
  const item = document.createElement("li");
  item.textContent = text;
  const del = document.createElement("button");
  del.textContent = "Delete";
  del.className = "delete";
  del.addEventListener("click", () => item.remove());
  item.appendChild(del);
  list.appendChild(item);
  input.value = "";
}
addBtn.addEventListener("click", addTask);
document.querySelectorAll(".delete").forEach(button => {
  button.addEventListener("click", () => button.parentElement.remove());
});
</script>
</body>
</html>
```

7. CODE EXPLANATION

Component	Purpose
getElementById()	Selects an HTML element by its id.
addEventListener()	Registers a function to respond to a user event.
createElement()	Creates a new DOM element dynamically.
textContent	Sets the visible text of a DOM element.
appendChild()	Adds a new node to the document tree.
remove()	Removes the selected task from the DOM.
trim()	Removes extra whitespace before validating input.

8. TEST CASES

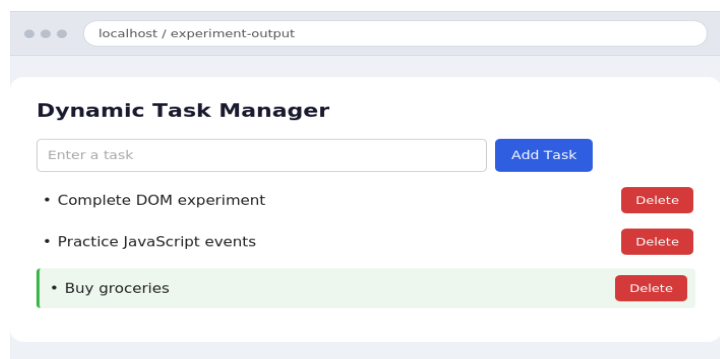
The following test cases were executed to validate the add-task and delete-task functionality of the webpage, with the corresponding output shown immediately below each case.

TC01 — Add a task with valid text

Input / Steps: Enter "Buy groceries" in input, click Add Task

Expected Output: New list item "Buy groceries" with a Delete button appears in the task list

Result: Pass

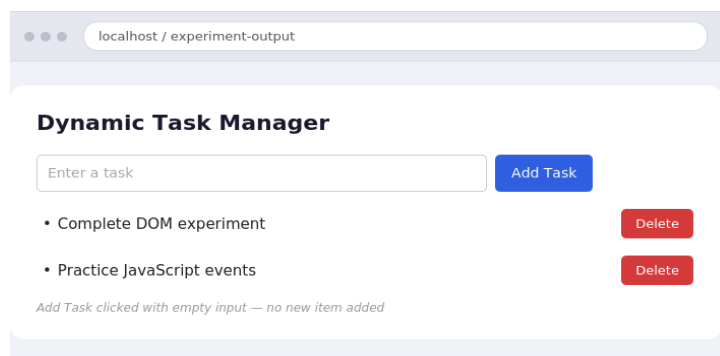


TC02 — Add a task with empty input

Input / Steps: Leave input field blank, click Add Task

Expected Output: No new task is added; list remains unchanged

Result: Pass

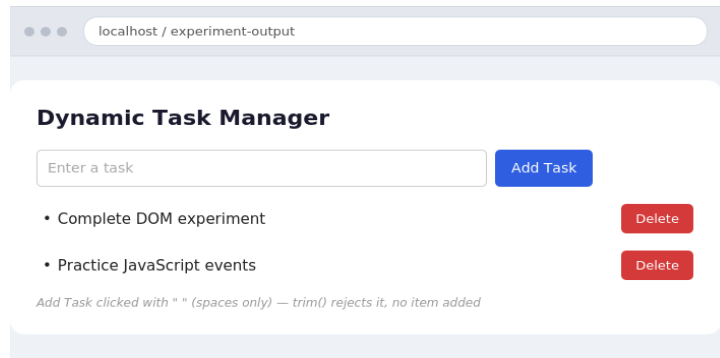


TC03 — Add a task with only spaces

Input / Steps: Enter " " (spaces only), click Add Task

Expected Output: trim() reduces input to empty string; no task is added

Result: Pass

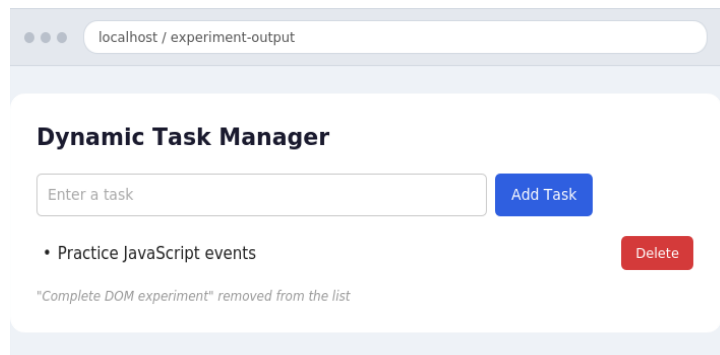


TC04 — Delete an existing static task

Input / Steps: Click Delete button next to "Complete DOM experiment"

Expected Output: That task is removed from the list immediately

Result: Pass

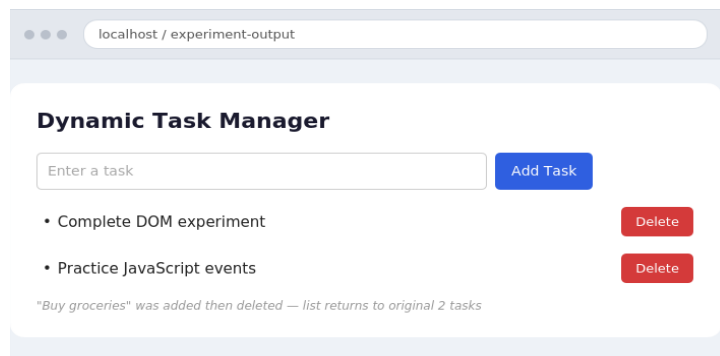


TC05 — Delete a dynamically added task

Input / Steps: Add a new task, then click its Delete button

Expected Output: The newly added task is removed from the DOM

Result: Pass

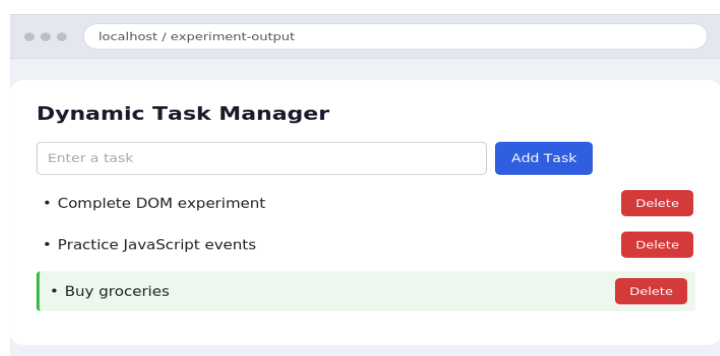


TC06 — Input field resets after adding

Input / Steps: Add a task and observe the input field

Expected Output: Input field becomes empty, ready for next entry

Result: Pass

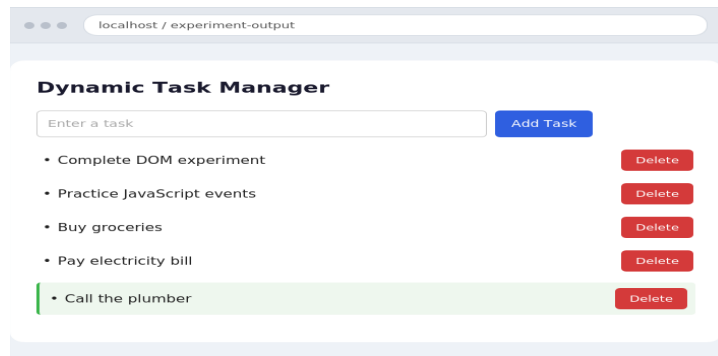


TC07 — Add multiple tasks sequentially

Input / Steps: Add 3 different tasks one after another

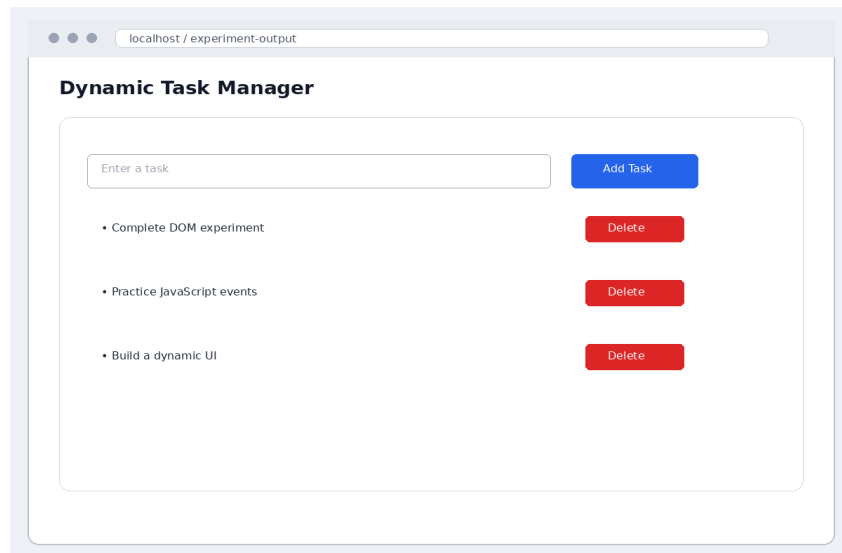
Expected Output: All 3 tasks appear in the list in the order added

Result: Pass



9. OUTPUT

Browser Output (Screenshot):



10. RESULT

Thus, a dynamic task-management webpage was successfully developed using JavaScript and DOM methods. Tasks can be added dynamically and individual tasks can be deleted through event handling. All test cases executed passed, confirming correct add and delete functionality.